

Python File Handling & Automation Task

Objective

This project demonstrates Python file handling, automation operations (rename, move, delete), and exception handling using try-except blocks.

Requirements Covered

- Read and write files
- Automate file operations
- Use try-except for error handling
- Explain code using comments

Python Code

```
# Import required modules
import os
import shutil

try:
    # STEP 1: Create and write data into a text file
    file = open("students.txt", "w")

    # Writing sample data
    file.write("101, Manoj, CSE\n")
    file.write("102, Ravi, ECE\n")
    file.write("103, Priya, IT\n")

    # Close the file
    file.close()

    print("Data written successfully.\n")

    # STEP 2: Read the file data
    file = open("students.txt", "r")

    print("Reading File Content:\n")
    content = file.read()
    print(content)

    file.close()

    # STEP 3: Rename the file
    os.rename("students.txt", "student_data.txt")
    print("File renamed successfully.\n")

    # STEP 4: Create a new folder
    folder_name = "Backup"

    # Check if folder already exists
    if not os.path.exists(folder_name):
        os.mkdir(folder_name)
        print("Backup folder created.\n")

    # STEP 5: Move the file into Backup folder
    shutil.move("student_data.txt", "Backup/student_data.txt")
    print("File moved successfully.\n")
```

```
# STEP 6: Delete the file
os.remove("Backup/student_data.txt")
print("File deleted successfully.\n")

except FileNotFoundError:
    print("Error: File not found.")

except PermissionError:
    print("Error: Permission denied.")

except Exception as e:
    print("An unexpected error occurred:", e)

finally:
    print("\nProgram execution completed.")
```

Expected Output

```
Data written successfully.

Reading File Content:

101, Manoj, CSE
102, Ravi, ECE
103, Priya, IT

File renamed successfully.

Backup folder created.

File moved successfully.

File deleted successfully.

Program execution completed.
```